

Project 1: Socket Programming

QAP – Quiz Application Protocol

จัดทำโดย นายชินภัทร ช่างวิเชียร 6710405290

บทนำ

โครงการนี้มีวัตถุประสงค์เพื่อศึกษาและพัฒนา application-layer protocol สำหรับระบบเกมตอบคำถามที่ทำงานบนเครือข่าย โดยใช้แนวคิดของ client-server architecture และการสื่อสารผ่าน socket programming ในระดับ Transport Layer และ Application Layer การพัฒนาระบบนี้มุ่งเน้นให้เห็นถึงความสำคัญของการออกแบบโปรโตคอลที่สามารถรองรับการส่งข้อมูลที่มีโครงสร้าง การจัดการ session state ระหว่าง client และ server และการตรวจสอบข้อมูลที่ถูกส่งมาอย่างถูกต้อง

ระบบที่พัฒนาขึ้นนี้เป็นเกมตอบคำถามที่ผู้เล่นสามารถเข้าร่วมผ่าน browser หรือผ่าน terminal client โดย server เป็นตัวกลางในการจัดการคำถาม การประเมินคะแนน และการแจก fragment ของ secret flag ให้แก่ผู้เล่นที่ตอบคำถามถูกต้องเพื่อเป็น path สำหรับเข้าถึงหน้า secret ได้ การออกแบบดังกล่าวช่วยให้เห็นว่าการสื่อสารในเครือข่ายไม่ได้เป็นเพียงแค่การส่งข้อความแบบ raw data แต่ต้องมีโครงสร้างและความหมายทางแอปพลิเคชันที่ชัดเจน

วัตถุประสงค์ของโปรแกรม

วัตถุประสงค์หลัก

เริ่มจากการพัฒนาโปรแกรมประเภท network application เพื่อให้ผู้ใช้งานสามารถทำงานร่วมกันระหว่าง client และ server ได้ผ่านระบบเครือข่าย โดยมีจุดประสงค์หลักดังนี้:

1. เพื่อศึกษาการออกแบบ request/response protocol ระดับแอปพลิเคชัน
2. เพื่อฝึกการใช้งาน TCP socket เพื่อเชื่อมต่อระหว่าง client และ server
3. เพื่อจัดการ session ของผู้เล่นแต่ละคนอย่างถูกต้องและต่อเนื่อง
4. เพื่อให้ server ตรวจสอบคำตอบและคำนวณคะแนนโดยอาศัยข้อมูลที่อยู่ฝั่ง server เท่านั้น
5. เพื่อให้ client ได้รับคำถามและส่งคำตอบกลับไปตามรูปแบบ protocol ที่กำหนด
6. เพื่อประยุกต์ใช้ custom header ในการส่ง metadata เช่น session ID และ flag fragment
7. เพื่อให้เห็นว่าความซับซ้อนของการสื่อสารเครือข่ายไม่ได้อยู่ที่ socket เพียงอย่างเดียว แต่ยังรวมถึง logic ของแอปพลิเคชันด้วย

โปรแกรมนี้นี้เป็นโปรแกรมอะไร

โปรเจกต์นี้เป็นแบบแอปพลิเคชันเกมตอบคำถาม (quiz challenge application) ที่ทำงานบนเครือข่ายแบบ client-server โดยมี server เป็นผู้ให้บริการข้อมูลคำถามและประเมินคำตอบของผู้เล่น ส่วน client เป็นส่วนที่ส่งคำถามและรับผลลัพธ์จาก server ระบบนี้มีลักษณะเป็น challenge-based application ที่มีจุดเด่นด้านความท้าทายและการจัดการ state ผ่าน network

โปรแกรมนี้ใช้ทำอะไร

โปรแกรมนี้ใช้ในการทำงานดังต่อไปนี้:

- Client เริ่มต้น session ใหม่เพื่อเริ่มเล่นเกม
- Server สร้าง session สำหรับผู้เล่นแต่ละคน
- Client ขอคำถามจาก server ตามลำดับที่ต้องการ
- Client ส่งคำตอบกลับไปยัง server
- Server ตรวจสอบถูกต้องของคำตอบ
- หากตอบถูก server จะมอบ fragment ของ secret flag ให้แก่ client
- Client รวบรวม fragment และต่อกันเป็น secret path
- เมื่อผู้เล่นทำคำถามครบทุกข้อแล้ว server คำนวณคะแนนและส่งผลลัพธ์กลับมา

ลักษณะของการทำงานนี้แสดงให้เห็นว่าโปรแกรมนี้เป็น network application ที่มีองค์ประกอบของแอปพลิเคชันการส่งข้อความ และมีการพึ่งพา session state และความถูกต้องของข้อมูลต่อเนื่อง

ลักษณะของแอปพลิเคชัน (Characteristics)

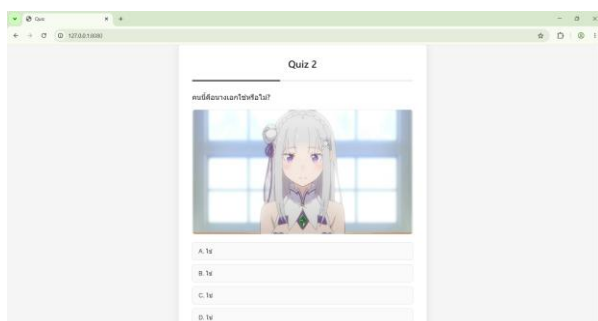


Figure 1: Browser interface for quiz play.

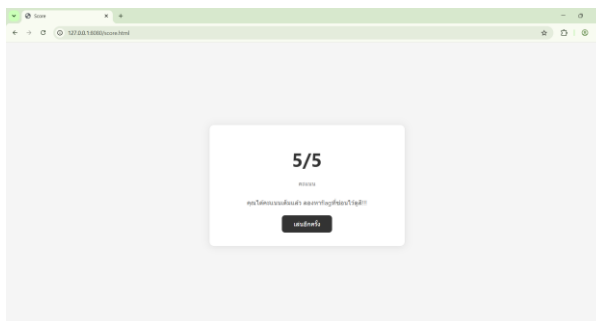


Figure 2: Browser interface score evaluation.

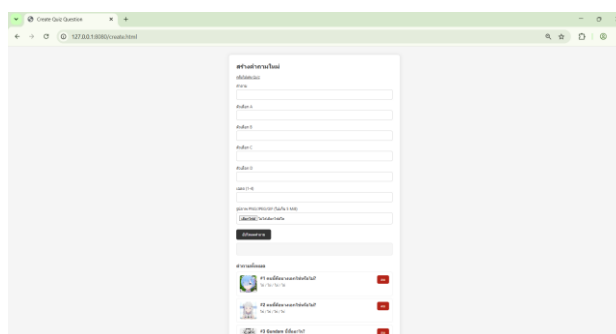


Figure 3: Browser interface for create quiz and delete quiz.

โปรแกรมนี้มีคุณลักษณะของ application ที่ชัดเจน ดังนี้:

Stateful application

ระบบนี้มีการเก็บ session ของผู้เล่นไว้บน server เช่น:

- snapshot ของคำถาม
- answer history
- score
- fragment ที่ได้รับจากการตอบถูก
- ความคืบหน้าของเกม

การมี session เป็นสิ่งจำเป็น เพราะ server ต้องทราบว่าผู้เล่นคนนี้เคยตอบคำถามข้อใดบ้าง และมีความคืบหน้ามากน้อยแค่ไหน เพื่อป้องกันการส่งคำตอบซ้ำและการคำนวณคะแนนผิดพลาด

Connection-oriented application

โปรแกรมนี้ใช้ TCP เป็นชั้น Transport Layer ทำให้การสื่อสารมีลักษณะ connection-oriented และ reliable

นั่นคือข้อมูลจะถูกส่งในลำดับที่ถูกต้องและมีความน่าเชื่อถือมากกว่าระบบที่ใช้ UDP

Request-response based communication

การสื่อสารระหว่าง client และ server เป็นไปตามรูปแบบ request-response ทุก request จะมี response ที่สอดคล้อง เช่น เมื่อ client ขอคำถาม server จะส่งข้อมูลคำถามกลับมา หรือเมื่อ client ส่งคำตอบ server จะตอบกลับว่าถูกหรือผิด

Custom protocol design

ระบบนี้ไม่ได้ใช้ protocol ที่เป็นมาตรฐานเต็มรูปแบบ แต่มีการออกแบบ protocol แบบกำหนดเองที่มีความคล้าย HTTP/1.1 อย่างมีนัยสำคัญ เช่น มี method, path, headers, content-length และ JSON body แต่ยังมี custom header เฉพาะเกม เช่น:

- X-Session-ID
- X-Flag-Fragment
- X-Flag-Fragment-Index
- X-Protocol

ดังนั้น protocol ของโปรเจกต์สามารถเรียกได้ว่าเป็น custom application-layer protocol

Security-aware design

Server มีการตรวจสอบข้อมูลที่ได้รับเพื่อป้องกันความไม่ถูกต้อง เช่น:

- ตรวจสอบว่ามี session ID หรือไม่
- ตรวจสอบว่า answer ซ้ำหรือไม่
- ตรวจสอบว่าค่าดัชนีคำตอบอยู่ในช่วงที่ถูกต้องหรือไม่
- ตรวจสอบขนาดของ request และ payload
- ตรวจสอบ MIME type และ magic bytes ของรูปภาพ
- ป้องกัน path traversal สำหรับไฟล์ static

Upload and media handling

ระบบนี้รองรับการอัปโหลดภาพประกอบคำถาม โดยใช้ base64 encoding ใน JSON payload และมีการตรวจสอบว่ารูปแบบภาพนั้นถูกต้องก่อนบันทึกลงบน server

Concurrency support

Server ใช้ ThreadPoolExecutor เพื่อรองรับการทำงานพร้อมกันจากหลาย client ต่าง ๆ และมีการใช้ lock ในการจัดการ session และ catalog ของคำถาม เพื่อป้องกัน race condition

Transport Layer Service Model ที่ใช้

การเลือกใช้ TCP

โปรเจกต์นี้เลือกใช้ TCP เป็น service model ของ Transport Layer เพราะมีความต้องการด้านความน่าเชื่อถือและการรักษาลำดับข้อมูลเป็นสำคัญ ตัวอย่างเช่น:

1. การส่งคำตอบจาก client ไปยัง server ต้องมีความถูกต้องและต้องไม่สูญหาย
2. การทำงานของเกมต้องมีลำดับของ request และ response ที่ถูกต้อง เช่น เริ่ม session ก่อนตอบคำถาม
3. Server ต้องมี session state ของผู้เล่นแบบต่อเนื่อง เพื่อให้ score และ fragment ถูกคำนวณอย่างถูกต้อง
4. ระบบมี keep-alive connection ที่พยายาม reuse socket ดังนั้น TCP จึงเหมาะสมกับลักษณะงานนี้มากกว่า UDP
5. Request และ response เป็นข้อความที่มีโครงสร้างและมี JSON payload ทำให้ TCP มีประสิทธิภาพในการจัดการ byte stream ได้ดี

การออกแบบ Application-Layer Protocol

ชื่อ protocol

ชื่อ protocol ที่ใช้ในงานนี้คือ: QAP (Quiz Application Protocol)

แนวคิดการออกแบบ

QAP ถูกออกแบบให้มีลักษณะคล้าย HTTP/1.1 แต่มีความก้าวหน้าและความเฉพาะสำหรับ game application เช่น:

- method-based request
- path-based resource
- headers สำหรับ session และ application metadata
- JSON payload
- response ที่มี status code และ status phrase

ความแตกต่างหลักคือ QAP ถูกสร้างขึ้นเพื่อรองรับความต้องการเฉพาะของเกมตอบคำถาม เช่น session, answer check, score computation และ secret fragment delivery

รูปแบบ request และ response

Request format

```
<METHOD> <PATH> HTTP/1.1
Host: 127.0.0.1:8080
X-Protocol: QAP
X-Session-ID: <session-id>
Connection: keep-alive
Content-Type: application/json
Content-Length: <bytes>

<JSON body>
```

Response format

```
HTTP/1.1 <STATUS> <PHRASE>
Content-Type: application/json; charset=utf-8
Content-Length: <bytes>
X-Protocol: QAP
X-Session-ID: <session-id>
Connection: keep-alive
X-Flag-Fragment: <fragment>
X-Flag-Fragment-Index: <index>

JSON body>
```

หน้าที่ของ custom headers

- X-Session-ID: ระบุตัวตนของ session
- X-Flag-Fragment: fragment ของ secret flag ที่ server แจกให้เมื่อตอบถูก
- X-Flag-Fragment-Index: ระบุตำแหน่งของ fragment ในลำดับที่ถูกต้อง
- X-Protocol: ระบุว่าข้อความนี้ใช้ protocol QAP

Request และ Response Messages ที่ใช้ใน protocol

START SESSION

Request

```
GET /api/start HTTP/1.1
Host: 127.0.0.1:8080
X-Protocol: QAP
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 145
X-Protocol: QAP
X-Session-ID: 6fa...c91
Connection: keep-alive

{"status":200,"message":"Quiz started","total_questions":5,
"question_ids":[1,2,3,4,5],"session_id":"6fa...c91"}
```

ความหมาย

- client เริ่มต้นเกม
- server สร้าง session ใหม่
- server ส่ง session ID กลับไปเพื่อใช้ในการ request ถัดไป

GET QUESTION

Request

```
GET /api/question/3 HTTP/1.1
Host: 127.0.0.1:8080
X-Protocol: QAP
X-Session-ID: 6fa...c91
Connection: keep-alive
```

Response

```
{
  "status": 200,
  "question": {
    "id": 3,
    "question": "Gundam นี้ชื่ออะไร?",
    "images": ["images/sample.png"],
    "choices": ["Barbatos", "Wing-Gundam", "Unicorn-Gundam", "Gundam"]
  }
}
```

ความหมาย

- server ส่งข้อมูลคำถามแบบ public โดยไม่เปิดเผยคำตอบที่ถูกต้องจริง
- client จะแสดงคำถามและตัวเลือกให้ผู้เล่นเลือกคำตอบ

ANSWER QUESTION**Request**

```
POST /api/answer/3 HTTP/1.1
Host: 127.0.0.1:8080
X-Protocol: QAP
X-Session-ID: 6fa...c91
Content-Type: application/json
Content-Length: 12
Connection: keep-alive

{"answer":2}
```

Response หากตอบถูก

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 80
X-Protocol: QAP
X-Session-ID: 6fa...c91
X-Flag-Fragment: m4st3r_
X-Flag-Fragment-Index: 3
Connection: keep-alive

{"status":200,"qap_status":201,"qap_phrase":"CORRECT","correct":true}
```

Response หากตอบผิด

```

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 80
X-Protocol: QAP
X-Session-ID: 6fa...c91
Connection: keep-alive

{"status":200,"qap_status":202,"qap_phrase":"WRONG","correct":false}

```

ความหมาย

- qap_status และ qap_phrase เป็น status ที่เฉพาะกับเกม
- หากตอบถูก server จะส่ง fragment ของ secret flag ให้ client

SCORE REQUEST**Request**

```

POST /api/score HTTP/1.1
Host: 127.0.0.1:8080
X-Protocol: QAP
X-Session-ID: 6fa...c91
Content-Type: application/json
Content-Length: 2
Connection: keep-alive

{}

```

Response

```

{
  "status": 200,
  "score": 5,
  "total": 5,
  "perfect": true,
  "fragments_earned": 5
}

```

ความหมาย

- server ตรวจสอบว่า player เข้าสู่สถานะ complete หรือยัง
- score ถูกคำนวณเชิง authoritative โดย server เท่านั้น

CREATE QUESTION

Request

```
{
  "question": "คำถามใหม่",
  "choices": ["A", "B", "C", "D"],
  "answer": 1,
  "image": {
    "filename": "picture.png",
    "content_type": "image/png",
    "data_base64": "iVBORw0KGgo..."
  }
}
```

Response

```
{
  "status": 201,
  "question_id": 6,
  "image_url": "/images/upload_xxx.png"
}
```

DELETE QUESTION

Request

```
DELETE /api/quiz/question/4 HTTP/1.1
Host: 127.0.0.1:8080
X-Protocol: QAP
Connection: keep-alive
```

Response

```
{
  "status": 200,
  "message": "Question deleted",
  "question_id": 4
}
```

Status Code และ Status Phrase ของ Protocol

HTTP-like status

Status Code	Phrase	ความหมาย
200	OK	การทำงานสำเร็จ
201	CREATED	สร้างข้อมูลสำเร็จ
400	BAD_REQUEST	Request มีรูปแบบไม่ถูกต้อง
401	SESSION_REQUIRED	ไม่มี session หรือ session ไม่ถูกต้อง
403	FORBIDDEN	การเข้าถึงไม่ได้รับอนุญาต
404	NOT_FOUND	ไม่มี resource หรือ question ที่ต้องการ
409	CONFLICT	answer ซ้ำ หรือไม่สามารถลบคำถามสุดท้ายได้
413	PAYLOAD_TOO_LARGE	body หรือ image ใหญ่เกินไปขนาด
415	UNSUPPORTED_MEDIA_TYPE	media type ไม่รองรับ
422	INCOMPLETE	ผู้เล่นยังตอบคำถามไม่ครบ
431	HEADERS_TOO_LARGE	header ใหญ่เกินไปกำหนด
500	INTERNAL_SERVER_ERROR	เกิดข้อผิดพลาดภายใน server

QAP-specific status

Code	Phrase	ความหมาย
201	CORRECT	ตอบคำถามถูก
202	WRONG	ตอบคำถามผิด

มีความสำคัญที่ช่วยให้ client และ server เข้าใจผลลัพธ์ของเกมแยกจาก status ของ HTTP โดยตรง

กลไก Secret Flag Fragmentation

แนวคิด

เมื่อผู้เล่นตอบคำถามถูก server จะมอบ fragment ของ secret flag ให้แก่ผู้เล่นหนึ่ง fragment ต่อคำถาม ตัวอย่าง flag คือ:

I_KNOW_YOU_YOU_KNOW_I_AND_I_NEVER_GONNA_GIVE_YOU_UP

server จะแบ่ง flag ออกเป็น N ส่วน ตามจำนวนคำถามใน session

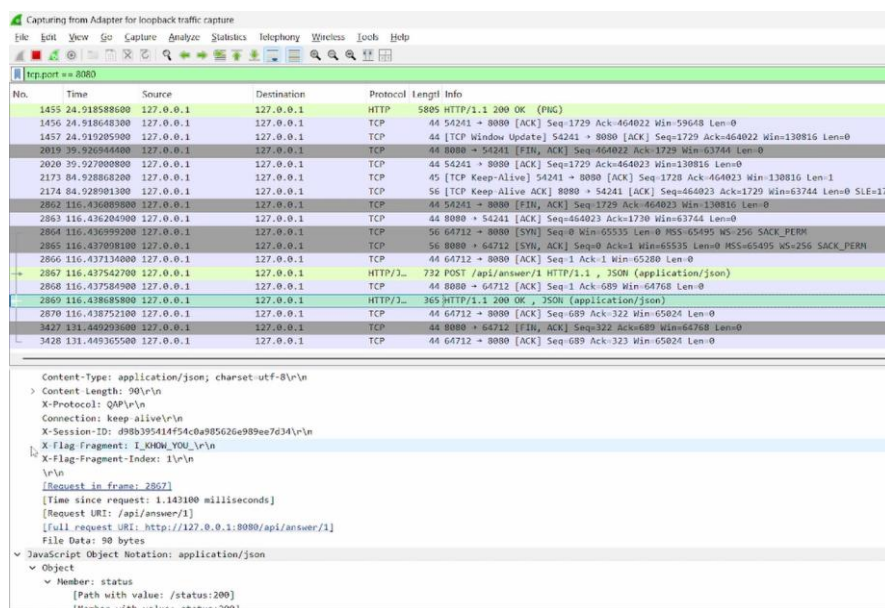


Figure 4: Wireshark capture showing QAP headers and flag fragments.

กระบวนการทำงาน

1. session ถูกสร้างขึ้นใน server
2. server แบ่ง flag ออกเป็น fragment ต่าง ๆ
3. เมื่อ client ส่งคำตอบถูก server จะส่ง fragment ที่เกี่ยวข้องกลับไป
4. client เก็บ fragment ตามลำดับ index
5. เมื่อเล่นครบทุกข้อ client จะเรียง fragment และต่อกันเป็น secret path

ประโยชน์

- เพิ่มความสนุกและความท้าทายด้านเกม
- แสดงให้เห็นถึงการออกแบบ protocol ที่มี stateful semantics
- สร้างประสบการณ์ที่แสดงให้เห็นว่าการสื่อสารในแอปพลิเคชันต้องมี metadata และ context ที่ชัดเจน

โครงสร้างของโปรเจกต์

Server

ไฟล์ `server.py` เป็นจุดศูนย์กลางของระบบที่รับผิดชอบงานหลักดังนี้:

- เปิด socket และรับ connection
- parse request
- validate session ID
- เช็กคำตอบและ score
- ให่ fragment ของ secret flag
- บันทึกและจัดการคำถาม
- ส่ง response กลับไปยัง client

Client

ไฟล์ `client.py` ทำหน้าที่สร้าง socket, ส่ง request, อ่าน response, จัดเก็บ fragment และแสดงข้อความและ status code

ที่เกิดขึ้นระหว่างการสื่อสารให้ผู้ใช้เห็นอย่างชัดเจน โค้ดนี้สอดคล้องกับเงื่อนไขที่ต้องการให้มีการพิมพ์ protocol messages และสถานะในแต่ละ request/response ออกมาได้อย่างชัดเจน